IPFS Gateway Migration Proof of Concept

Content-Addressed Storage with QUIC Server-Side Migration

PoC Documentation • July 2026

This document describes a proof-of-concept demonstrating QUIC server-side connection migration between two IPFS gateways. Both gateways pin the same content (identified by CID), and the primary gateway advertises the preferred gateway's address during the QUIC handshake. IPFS's content-addressing model guarantees that content served by either gateway is cryptographically identical, making IPFS a natural fit for migration scenarios. The PoC runs on our existing four-machine testbed using modified Neqo binaries. We cover IPFS fundamentals (CIDs, Merkle DAGs, Bitswap, IPNI), the role of libp2p and QUIC in the IPFS stack, gateway vs. peer-to-peer considerations, and scalability implications for production gateway clusters.

Table of Contents

1. IPFS Fundamentals

IPFS (InterPlanetary File System) is a peer-to-peer, content-addressed storage and distribution protocol. Unlike traditional web hosting where content is identified by its location (URL), IPFS identifies content by **what it is** -- using cryptographic hashes.

1.1 Content Identifiers (CIDs)

A CID (Content Identifier) is a self-describing, cryptographic hash of a piece of content. CIDs use multihash encoding, which embeds the hash algorithm and digest length in the identifier itself. CIDv0 uses bare SHA-256 multihash (starting with Qm...), while CIDv1 adds multibase and multicodec prefixes for extensibility (starting with bafy...).

```
CID = multibase + multicodec + multihash(content)
```

The critical property: if two nodes have content with the same CID, that content is byte-for-byte identical. This is enforced by the hash function -- SHA-256 collision resistance ensures no two different inputs produce the same CID.

1.2 Merkle DAGs

IPFS organizes data as a Merkle DAG (Directed Acyclic Graph). Large files are split into blocks (default 256 KiB), each block gets its own CID, and a root node links to all block CIDs. The root CID recursively captures the integrity of the entire tree. Modifying a single byte in any block changes all CIDs from that block up to the root.

```
File -> [Block 0 (CID_0)] [Block 1 (CID_1)] ... [Block N (CID_N)]
        Root CID = hash(link: CID_0, link: CID_1, ..., link: CID_N)
```

This Merkle tree structure provides two benefits for gateway migration: (1) partial verification -- a client can verify individual blocks without downloading the entire file, and (2) deduplication -- identical blocks across files share the same CID and storage.

1.3 Content Addressing vs. Location Addressing

Traditional web: **https://example.com/file.txt** identifies WHERE to get the file. If the server changes the file, the URL stays the same but returns different content. IPFS: **/ipfs/QmT78zSuBmuS4z925WZfrq...** identifies WHAT the file is. Any node that has this CID serves the exact same content. This is why IPFS is a natural fit for server migration -- the preferred server is guaranteed to serve the same data.

Content addressing eliminates the fundamental trust problem in server migration: there is no need to trust that the preferred server has the correct data. The CID IS the verification.

2. IPFS Implementations

Several IPFS implementations exist, each targeting different use cases:

Implementation	Description	Use Cases
Kubo (Go)	Reference implementation. Most mature, feature-complete. Default for servers and gateways. Used in this PoC.	Production gateways, full nodes, pinning services

Implementation	Description	Use Cases
Iroh (Rust)	High-performance implementation by n0. Focuses on efficiency and embeddability. Does not implement full IPFS spec.	Embedded systems, high-throughput applications
Helia (JavaScript)	Successor to js-ipfs. Runs in Node.js and browsers. Modular architecture with pluggable components.	Browser-based dApps, lightweight nodes
Nabu (Java)	JVM-based implementation. Targets enterprise Java ecosystems.	Enterprise integration, JVM applications

This PoC uses **Kubo** (formerly go-ipfs) because it is the reference implementation with the most complete feature set, the widest deployment base, and built-in HTTP gateway functionality. Kubo's gateway listens on port 8080 by default and serves content over plain HTTP, which our modified Neqo server proxies to clients over HTTP/3.

This PoC uses Kubo (Go), the reference IPFS implementation. Kubo's built-in HTTP gateway (port 8080) is proxied by our Neqo HTTP/3 server.

3. libp2p and QUIC in the IPFS Stack

IPFS nodes communicate using libp2p, a modular networking stack that abstracts transport, peer discovery, and connection multiplexing. Since Kubo v0.18 (November 2022), QUIC is the **default transport** for peer-to-peer connections, replacing the older TCP+Noise stack.

3.1 libp2p Transport Layer

libp2p supports multiple transports simultaneously. A Kubo node typically listens on:

Multiaddr	Transport
/ip4/0.0.0.0/udp/4001/quic-v1	QUIC v1 (default, preferred)
/ip4/0.0.0.0/udp/4001/quic-v1/webtransport	WebTransport over QUIC
/ip4/0.0.0.0/tcp/4001	TCP + Noise (fallback)
/ip6/::/udp/4001/quic-v1	QUIC v1 over IPv6

QUIC provides several advantages for IPFS: (1) built-in encryption (TLS 1.3), eliminating the need for a separate Noise handshake; (2) multiplexed streams without head-of-line blocking; (3) faster connection establishment (0-RTT or 1-RTT); (4) native connection migration support (RFC 9000 Section 9).

3.2 PeerID and TLS Certificates

Each IPFS node has a PeerID derived from its public key (typically Ed25519). During the QUIC/TLS handshake, the node presents a self-signed X.509 certificate containing a libp2p extension (OID 1.3.6.1.4.1.53594.1.1) that embeds its public key. The connecting peer verifies that the certificate's embedded key matches the expected PeerID.

This creates a fundamental challenge for peer-to-peer migration: migrating a QUIC connection to a different peer means the new peer has a different PeerID and certificate. The client would detect the identity change and reject the connection. This is why our PoC operates at the gateway layer, where standard TLS certificates (not PeerID-based) are used.

Peer-to-peer migration is blocked by PeerID authentication. Gateway migration (HTTP/3 with standard TLS certs) sidesteps this limitation.

4. Gateway vs. Peer-to-Peer Migration

There are two conceptually different levels at which QUIC migration could apply to IPFS:

4.1 Gateway Migration (This PoC)

IPFS gateways are HTTP servers that bridge IPFS and the traditional web. They accept standard HTTP requests (e.g., GET /ipfs/QmT78z...) and return the corresponding IPFS content. Gateway migration is feasible today because:

- Gateways use standard TLS certificates (not PeerID-based), so migration does not trigger identity verification failures.

- Gateways are stateless with respect to content: any gateway that pins the CID can serve it.
- Production gateways already operate as clusters behind load balancers -- migration formalizes this at the QUIC protocol layer.
- No changes to IPFS clients (browsers) are required -- they use standard HTTP/3.

4.2 Peer-to-Peer Migration (Future Work)

Migrating native IPFS peer-to-peer connections is significantly harder because:

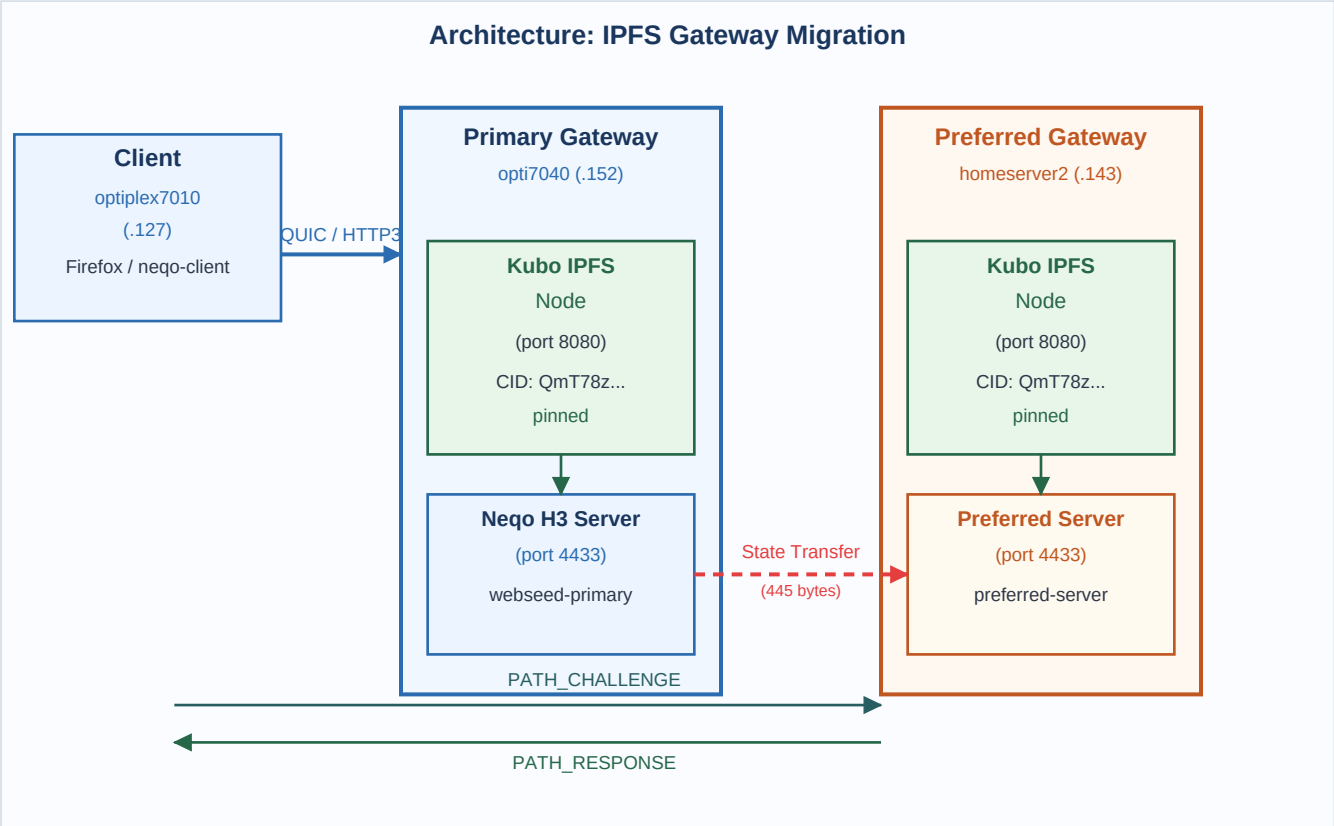
- PeerID is cryptographically bound to the TLS certificate. Migrating to a different peer changes the identity.
- Active Bitswap sessions carry complex state (want-lists, block requests, session context) that would need serialization.
- libp2p's connection security model assumes end-to-end identity verification, which migration inherently violates.
- The receiving peer would need the sending peer's private key to maintain identity, which defeats the security model.

Gateway migration is feasible TODAY with no protocol changes. Peer-to-peer migration requires fundamental libp2p authentication changes.

5. Architecture

The PoC uses our existing four-machine testbed on the same LAN (141.217.168.x):

Role	Machine	Components
Primary Gateway	opti7040 (.152)	Kubo IPFS daemon + Neqo HTTP/3 server (webseed-primary)
Preferred Gateway	homeserver2 (.143)	Kubo IPFS daemon + preferred-server binary
Client	optiplex7010 (.127)	Firefox browser or neqo-client
Redis (optional)	Proxmox VM (.200)	State transfer backend (Redis KV or Pub/Sub)



Both the primary and preferred gateways run a local Kubo IPFS node, each pinning the same test content. The primary gateway fetches content from its local IPFS HTTP gateway (localhost:8080) and serves it to clients over HTTP/3 via the webseed-primary binary. During the QUIC handshake, the primary advertises the preferred gateway's address (.143) using the preferred_address transport parameter.

After the handshake, the primary exports 445 bytes of migration state (TLS secrets, connection IDs, client address, version, packet numbers) and transfers it to the preferred server. The preferred server imports this state and handles the client's PATH_CHALLENGE/PATH_RESPONSE validation, completing the migration transparently.

6. How It Works

1. Pin content on both nodes: Both IPFS nodes pin the same content, identified by a CID (Content Identifier). The CID is a cryptographic hash of the content. Pinning ensures the content is retained locally and not garbage-collected.

2. Primary fetches from local IPFS: The primary server's webseed-primary binary fetches content from the local IPFS gateway at localhost:8080/ipfs/<CID>. This is a simple HTTP GET to the Kubo gateway running on the same machine.

3. Serve via HTTP/3: The primary serves the fetched content to the client over HTTP/3 using the modified Neqo stack. The client (Firefox) sees a standard HTTPS page.

4. Advertise preferred address: During the QUIC handshake, the primary includes preferred_address = 141.217.168.143 in its transport parameters (RFC 9000 Section 9.6).

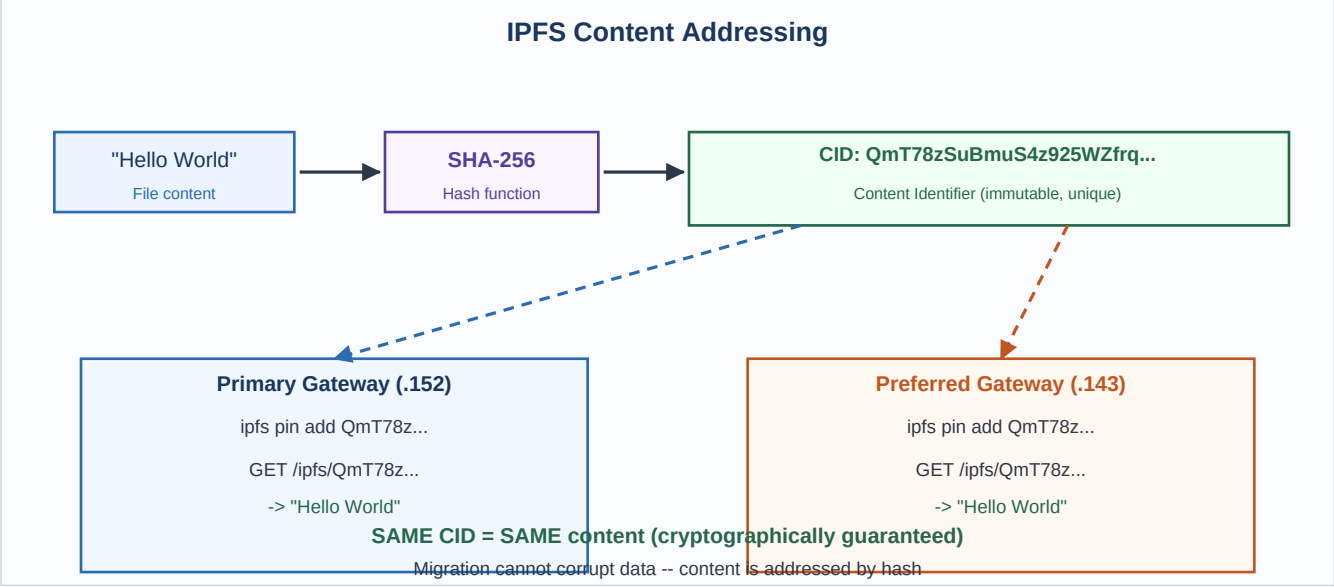
5. Export and transfer state: The primary exports migration state (445 bytes: TLS secrets, CIDs, client address, version, packet numbers) and sends it to the preferred server via the configured state transfer backend (TCP push, HTTP pull, Redis KV, or Redis Pub/Sub).

6. Client validates new path: The client sends PATH_CHALLENGE to the preferred server. The preferred server, having imported the crypto state, decrypts the challenge and sends an encrypted PATH_RESPONSE back. The client validates the response and migrates.

7. Content integrity guaranteed: Because both gateways pin the same CID, the content is cryptographically identical. IPFS guarantees this by design -- the CID IS the hash of the content.

Content integrity is GUARANTEED by IPFS: the CID is a cryptographic hash of the content. Same CID = same content, regardless of which gateway serves it.

7. Content Verification



Content verification in IPFS is fundamentally different from traditional web systems. In a traditional CDN, verifying that the preferred server has the correct content requires external mechanisms: SHA-256 checksums, Subresource Integrity (SRI) hashes, or certificate transparency logs. These are bolt-on verification layers that

must be explicitly implemented and maintained.

With IPFS, verification is **intrinsic**. The CID itself is the verification. When a client requests /ipfs/QmT78z..., any node that responds must provide content whose SHA-256 hash matches the CID. If the content does not match, the IPFS node rejects it. No additional verification infrastructure is needed.

For gateway migration, this means: (1) both gateways pin the same CID, so both serve identical content; (2) if either gateway's content is corrupted, the CID mismatch is detectable; (3) the client can independently verify content integrity by hashing the received data and comparing it to the CID.

IPFS provides zero-configuration content verification. No SRI hashes, no checksums, no certificate transparency. The CID IS the verification.

8. Bitswap Protocol

Bitswap is the data exchange protocol used by IPFS nodes to request and send blocks. It operates as a block-level barter system: nodes maintain want-lists (blocks they need) and exchange blocks with connected peers.

8.1 How Bitswap Works

1. A node adds a CID to its want-list (WANT_HAVE or WANT_BLOCK message).
2. Connected peers check if they have the requested block.
3. If a peer has the block, it sends a HAVE response (or the block itself for WANT_BLOCK).
4. The requesting node sends a BLOCK request to the peer that has it.
5. The peer sends the block data, and the requesting node verifies it against the CID.
6. The block is stored locally and can now be served to other peers.

8.2 Bitswap and Gateway Migration

In the gateway model, Bitswap is used **between IPFS nodes** to ensure both gateways have the content pinned. The gateway-to-client communication uses standard HTTP/3, not Bitswap. This separation is key: the complex Bitswap session state does not need to be migrated. Only the QUIC/TLS state (445 bytes) is transferred.

If both gateways are connected as IPFS peers, pinning content on the primary automatically makes it available to the preferred gateway via Bitswap. The preferred gateway can fetch blocks it does not yet have directly from the primary.

Bitswap handles content replication between IPFS nodes. Gateway migration only transfers QUIC state (445 bytes), not Bitswap sessions.

9. IPNI (InterPlanetary Network Indexer)

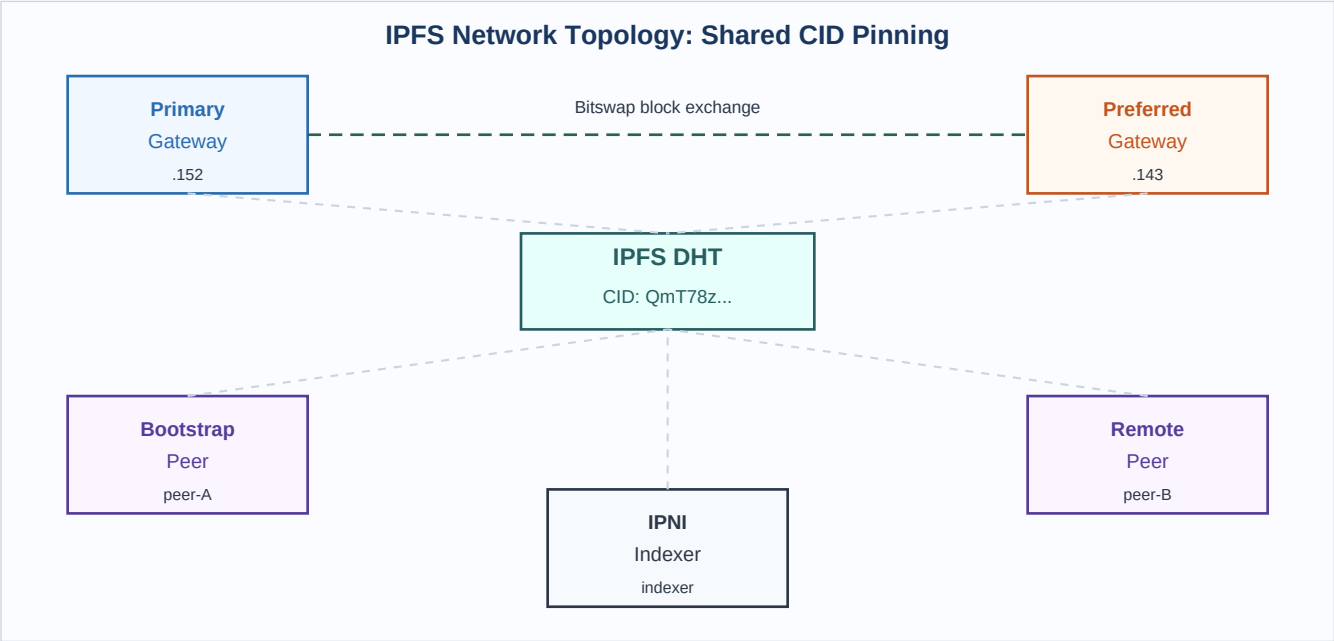
IPNI is the content routing system that helps clients find which IPFS nodes provide specific content. When a node pins content, it advertises the CID to IPNI indexers. Clients query the indexer to find providers before connecting to them.

9.1 Content Routing Flow

1. A node pins content and publishes an advertisement to IPNI (via HTTP or gossipsub).
2. IPNI indexers (e.g., cid.contact) ingest the advertisement and index the CID-to-provider mapping.
3. A client wanting content queries the indexer: "Who has CID QmT78z...?"
4. The indexer returns a list of providers (PeerIDs and multiaddrs).
5. The client connects to a provider and fetches the content via Bitswap or HTTP.

9.2 IPNI and Gateway Migration

In the gateway migration context, IPNI plays a supporting role. Both gateways advertise the same CID to the indexer. A client that discovers one gateway through IPNI and connects via HTTP/3 can be seamlessly migrated to the other gateway, which also advertises the same CID. The IPNI indexer can also be used to select the optimal preferred server based on geographic proximity or load.


IPFS Network Topology: Shared CID Pinning
Primary
Gateway
.152
Bitswap block exchange
Preferred
Gateway
.143
IPFS DHT
CID: QmT78z...
Bootstrap
Peer
peer-A
IPNI
Indexer
indexer
Remote
Peer
peer-B

10. Production IPFS Gateways

Several production IPFS gateways serve content to millions of users. These gateways already operate as multi-server clusters, making them natural candidates for QUIC server-side migration:

Gateway	Operator	Description
ipfs.io	Protocol Labs	Reference gateway. Multi-region deployment with geographic routing via anycast DNS.
dweb.link	Protocol Labs	Trustless gateway supporting verifiable responses (IPFS Trustless Gateway spec).
cf-ipfs.com	Cloudflare	Cloudflare IPFS Gateway. Runs on Cloudflare's edge network across 300+ cities worldwide.
w3s.link	web3.storage	Gateway for web3.storage pinning service. Backed by Elastic IPFS infrastructure.
nftstorage.link	NFT.Storage	Specialized gateway for NFT metadata and assets. High-availability with global distribution.

These gateways already use load balancing and geographic routing to distribute traffic. QUIC server-side migration would provide a protocol-native mechanism for mid-connection server switching, eliminating the need for DNS-based or HTTP-redirect-based failover. Benefits include: zero extra round-trips (unlike HTTP 302 redirects), preserved connection state (unlike DNS failover), and transparent operation (the client does not need to re-establish TLS).

Production IPFS gateways already operate as clusters. QUIC migration adds protocol-native, zero-round-trip server switching to their existing infrastructure.

11. Setup Requirements

11.1 Kubo Installation

Kubo is distributed as a single binary. Install on both gateway machines:

```
wget https://dist.ipfs.tech/kubo/v0.32.1/kubo_v0.32.1_linux-amd64.tar.gz
tar xzf kubo_v0.32.1_linux-amd64.tar.gz
sudo mv kubo/ipfs /usr/local/bin/
ipfs init
```

11.2 Gateway Configuration

Configure the IPFS gateway to listen on the appropriate interface:

```
ipfs config Addresses.Gateway /ip4/127.0.0.1/tcp/8080
ipfs config --json Gateway.PublicGateways '{}'
```

11.3 Peer Connection

Connect the two IPFS nodes so they can exchange blocks via Bitswap:

```
# On primary (.152):
ipfs swarm connect /ip4/141.217.168.143/udp/4001/quic-v1/p2p/<PEER_ID_143>

# Verify connection:
ipfs swarm peers | grep 141.217.168.143
```

11.4 Pin Test Content

Pin the same content on both nodes. The CID ensures both serve identical data:

```
# On primary (.152):
echo "Hello World" | ipfs add -Q
# Returns: QmT78zSuBmuS4z925WZfrqQ1qHaJ56DQaTfyMUF7F8ff5o

# On preferred (.143):
ipfs pin add QmT78zSuBmuS4z925WZfrqQ1qHaJ56DQaTfyMUF7F8ff5o
```

11.5 Verify Content

Verify both gateways serve the same content:

```
curl http://localhost:8080/ipfs/QmT78zSuBmuS4z925WZfrqQ1qHaJ56DQaTfyMUF7F8ff5o
# Should print: Hello World
```

12. How to Run

Three shell scripts automate the IPFS environment setup, testing, and teardown:

12.1 Setup IPFS on both servers

Run from the client machine (optiplex7010). This installs Kubo, initializes repositories, starts the IPFS daemons, and pins test content on both gateway servers.

```
./setup_ipfs.sh
```

12.2 Start the preferred server

On homeserver2 (.143), start the preferred server binary. This listens for incoming migration state on port 9999 and serves HTTP/3 on port 4433.

```
preferred-server 141.217.168.143:4433 9999
```

12.3 Start the primary server

On opti7040 (.152), start the webseed-primary binary. It fetches content from the local IPFS gateway and serves it over HTTP/3, advertising .143 as the preferred address.

```
webseed-primary 0.0.0.0:4433 141.217.168.143:4433 /tmp/ipfs_serve_content.bin
```

12.4 Test the migration

Run the test script from the client machine, or open Firefox and navigate to https://141.217.168.152:4433/. The page should load via HTTP/3 with a transparent migration to the preferred server.

```
./test_ipfs_migration.sh
```

12.5 Teardown

Stops the IPFS daemons on both servers and cleans up temporary files. Does not uninstall Kubo or remove the IPFS repositories.

```
./teardown_ipfs.sh
```

13. Scenarios and Scalability

13.1 Gateway Load Balancing

When an IPFS gateway becomes overloaded, it can use QUIC preferred_address to migrate incoming connections to a less-loaded gateway that pins the same content. Unlike DNS-based load balancing, this happens mid-connection without requiring a new TLS handshake, reducing latency and preserving connection state.

13.2 Gateway Maintenance

Before shutting down a gateway node for maintenance, existing connections can be gracefully drained to the preferred server. New connections are directed to the preferred server immediately via preferred_address, while existing connections complete their current transfers before migrating. This enables zero-downtime

maintenance of gateway infrastructure.

13.3 Geographic Optimization

An IPFS gateway can accept initial connections (via anycast or DNS) and then redirect clients to a geographically closer gateway. After the initial handshake reveals the client's IP address, the primary gateway selects the optimal preferred server based on network topology. The client migrates transparently, improving latency for subsequent requests without requiring client-side changes.

13.4 IPFS Gateway Clusters with QUIC Migration

In a production deployment, IPFS gateway clusters can use QUIC migration for load distribution across multiple backend servers. The architecture is:

- A pool of N gateway servers, each running Kubo and pinning popular content.
- An entry gateway accepts initial connections and performs the QUIC handshake.
- Based on load metrics, the entry gateway selects a backend server and advertises its address via preferred_address.
- Migration state (445 bytes) is transferred via Redis KV (recommended for clusters) or HTTP pull.
- The backend server handles subsequent requests, serving content from its local IPFS node.
- If the backend becomes overloaded, it can chain another migration to a different backend.

Scalability advantage: IPFS's content-addressing means any server pinning the CID can serve as a migration target. No session affinity or content synchronization logic is needed -- the CID guarantees content consistency.

14. Limitations

14.1 Peer Identity

IPFS nodes have PeerIDs that are cryptographically tied to their TLS certificates. Migration between different peers requires changes to libp2p's authentication layer, since the receiving node has a different identity. This PoC operates at the gateway layer (HTTP/3), where PeerID authentication is not enforced for HTTP clients.

Peer Identity: IPFS PeerIDs are tied to TLS certificates. Migration between different peers requires libp2p authentication changes.

14.2 Bitswap Streams

Active Bitswap streams (the protocol IPFS uses for peer-to-peer block exchange) carry complex state including want-lists, block requests, and session information. This state cannot be easily serialized and transferred between nodes. Migration of native IPFS peer-to-peer connections would require significant Bitswap protocol modifications.

14.3 Gateway-Mode Only

This PoC operates in gateway mode -- serving IPFS content over HTTP/3 to standard clients (browsers). It does not attempt to migrate native peer-to-peer IPFS transport connections (which use libp2p + QUIC). The gateway-mode approach is practical because it requires no changes to IPFS clients and leverages standard HTTP/3 semantics.

This PoC uses gateway-mode (HTTP/3), not native peer-to-peer IPFS transport. Native IPFS migration would require libp2p and Bitswap protocol changes.

14.4 Mutable Content (IPNS)

IPFS CIDs are immutable -- they always refer to the same content. Mutable references (IPNS names) add a layer of indirection: an IPNS name resolves to a CID, and the mapping can be updated. During migration, both gateways must resolve the same IPNS name to the same CID, which requires IPNS record propagation. Stale IPNS records could cause the preferred gateway to serve an older version of the content.

14.5 Large Content and Partial Pinning

If the preferred gateway has not fully pinned the content (e.g., it is still fetching blocks via Bitswap), requests for unpinned blocks will result in Bitswap lookups, increasing latency. For production deployments, the preferred gateway should pre-pin popular content or use the primary gateway as a Bitswap peer for fast block retrieval.

15. Comparison: IPFS vs Traditional CDN

The following table compares IPFS gateway migration with traditional CDN approaches across key dimensions relevant to server-side connection migration:

Feature	Traditional CDN	IPFS Gateway + Migration
Content ID model	Location-based (URL)	Content-based (CID)

Feature	Traditional CDN	IPFS Gateway + Migration
Content verification	None (trust server)	CID hash (cryptographic)
Content availability	Operator-managed replication	P2P replication (Bitswap)
Migration safety	Data could differ	Guaranteed identical (same CID)
Integrity after migration	Requires SRI/checksums	Automatic (CID = hash)
Server discovery	DNS / load balancer	IPNI indexer + DHT
Client changes needed	None	None (standard HTTP/3)
Content deduplication	Manual / CDN-specific	Automatic (Merkle DAG)
Partial verification	Not supported	Per-block CID verification
Multi-provider	CDN edge locations	Any pinning node

The key advantage of IPFS for migration is **content verification**: with traditional CDNs, there is no protocol-level guarantee that the preferred server will serve the same content as the primary. With IPFS, the CID cryptographically binds the content to its hash. If the preferred gateway returns different data, the CID will not match, and the client (or an intermediate verification layer) can detect the discrepancy.

IPFS + QUIC migration provides cryptographic content integrity guarantees that traditional CDNs cannot match. The CID ensures identical content across all gateways.